24/9/26

Practical -7

SLIDING WINDOW PROTOCOL

Aim: To write a program to implement flow control at data link layer using sliding window protocol. Simulate the flow of frames from one node to another

sender.py:

Program:

```
import time
import os
print("Sliding Window Protocol - Sender")
window_size = int(input("Enter window size: "))
if window_size <= 0:
    print("Invalid window size!")
    exit()
message = input("Enter text message: ")
if not message:
    print("Message cannot be empty!")
    exit()
frames = []
for i, character in enumerate(message):
    frames.append({
        "frame_no": i,
        "data": character

    })
print("\nFrames created:")
for frame in frames:
    print(f"Frame No: {frame['frame_no']} Data: {frame['data']}")
total_frames = len(frames)
base = 0
```

```
while base < total_frames:
    end = min(base + window_size, total_frames)
    print(f"Sending frame {base} to {end-1}")
    with open("Sender_Buffer","w") as file:
        file.write("Frame_No Data\n")
        for i in range(base, end):
            file.write(
                f"{frames[i]['frame_no']} {frames[i]['data']}
            )
    print("\nSender_Buffer:")
    for i in range(base, end):
        print(
            f"Frame No: {frames[i]['frame_no']}\t"
            f"Data: {frames[i]['data']}"
        )
    print("\nWaiting for acknowledgement...")
    time.sleep(3)
    if not os.path.exists("Receiver_Buffer"):
        print("\nReceiver_Buffer not found.")
        print("Run receiver.py and process the
                                    Sender_Buffer.")
        input("Press 'Enter' after receiver has
                                    processed the buffer...")

    try:
        with open("Receiver_Buffer","r") as file:
            response = file.readline().strip()

    except FileNotFoundError:
        print("Receiver_Buffer not found!")
        continue
```

```
parts = response.split()
if len(parts) != 2:
    print("Invalid Receiver_Buffer format!")
    continue

ack_type = parts[0]

try:
    ack_number = int(parts[1])
except ValueError:
    print("Invalid ACK/NACK number!")
    continue
print(f"\n Receiver response: {ack_type} {ack_number}")

if ack_type == "ACK":
    if ack_number == end:
        print("ACK is as expected.")
        print("Sliding window moves forward.")
        base = ack_number
    elif base < ack_number <= end:
        print("Cumulative ACK received.")
        print(f"Moving window to frame
                                {ack_number}.")
        base = ack_number;
    else:
        print("Unexpected ACK.")
        print("Resending current window.")
elif ack_type == "NACK":
    print(f"NACK received for frame {ack_number}.")
    print("Retransmitting frames from that frame.")
    if base <= ack_number < end:
        base = ack_number
    else:
        print("Invalid NACK number.")
        print("Resending current window.")
```

```
        else:
            print("Invalid response received!")
        time.sleep(1)
print("All frames transmitted successfully!")
print(f"Original Message: {message}")
```

Input:
Sliding Window Protocol - Sender
Enter window size: 3
Enter text message: HELLO

Output:

Sending frames 0 to 2
Sender_Buffer:
Frame No: 0 Data: H
Frame No: 1 Data: E
Frame No: 2 Data: L
Waiting for acknowledgement...
Receiver response: ACK 3
ACK is as expected.
Sliding window moves forward.

Sending frames 3 to 4
Sender_Buffer:
Frame No: 3 Data: L
Frame No: 4 Data: O
Waiting for acknowledgement...
Receiver response: ACK 5
ACK is as expected.
Sliding window moves forward.
All frames transmitted successfully!
Original Message: HELLO

receiver.py

Program:

```
import time
import os
SENDER_BUFFER = "Sender_Buffer.txt"
RECEIVER_BUFFER = "Receiver_Buffer.txt"
print("Sliding Window Protocol - Receiver")
print("Waiting for frames ...\n")
expected_frames = 0
last_content = ""
while True:
    if not os.path.exists(SENDER_BUFFER):
        time.sleep(1)
        continue
    with open(SENDER_BUFFER, "r") as file:
        content = file.read()
    if content == last_content or content.strip() == "":
        time.sleep(1)
        continue
    last_content = content
    lines = content.strip().splitlines()
    error = False
    nack_number = expected_frames
    print("\nSender_Buffer received:")
    for line in lines:
        try:
            parts = line.split(",")
            frame_part = parts[0].strip()
            data_part = parts[1].strip()
            frame_no = int(frame_part.split(":")[1].
                           strip())
            data = data_part.split(":")[1].strip()
```

```
except (IndexError, ValueError):
    print("Invalid frame format!")
    error = True
    nack_number = expected_frame
    break
print("Frame No:", frame_no, "Data:", data)
if frame_no == expected_frame:
    expected_frame += 1
else:
    print("Unexpected Frame No:", frame_no)
    print("Expected Frame No:", expected_frame)
    error = True
    nack_number = expected_frame
    break
print("Frame No:", frame_no, "Data:", data)
if frame_no == expected_frame:
    expected_frame += 1
else:
    print("Unexpected Frame No:", frame_no)
    print("Expected Frame No:", expected_frame)
    error = True
    nack_number = expected_frame
    break
with open(RECEIVER_BUFFER, "w") as file:
    if error:
        file.write(f"NACK:{nack_number}")
        print("NACK sent:", nack_number)
    else:
        file.write(f"ACK:{expected_frame}")
        print("ACK sent:", expected_frame)
time.sleep(1)
```

Output:

Sliding Windaw Protocol - Receiver

Waiting for frames...

Sender - Buffer received:

Frame No: 0 Data: H

Frame No: 1 Data: E

Frame No: 2 Data: L

ACK sent: 3

Sender - Buffer received:

Frame No: 3 Data: L

Frame No: 4 Data: O

ACK sent: 5

Result:

Thus, the Sliding Window Protocol was successfully implemented, and the flow of frames with ACK/NACK was simulated successfully.